

System design patterns

06/06/2026 18:00:17

<https://claude.ai/chat/e505d378-2d88-425f-8f3a-0aface3313b6>

Write 1 page per topic: Load Balancer (mention types of load balancer as wel...

System Design Interview Notes --- ## 1. Load Balancer **What it is:** A loa...

webhooks basics Grpc Basics exception handling patterns performance tuni...

System Design — Round 2 Notes --- ## 1. Webhooks Basics **What it is:** ...

How to do db partition what are the ways

Database Partitioning — Complete Guide --- ## What is Partitioning?...



Write 1 page per topic:
Load Balancer (mention types of load balancer as well)
Database (SQL vs NoSQL)
Cache (Redis basics) (mention types of cache as well)
Message Queue (Kafka / RabbitMQ idea)
API Gateway
Microservices (Monolithic vs Microservices)
👉 **For each:**
what it is
why we use it
simple example

"How does caching work?"

"Why use message queue?"

"and when there is kafka for event processing again why we should implement any Queue here whats the need"

"SQL vs NoSQL difference?"

"How to scale system?"

👉 Keep answers 5–10 lines max

explain these 2

Redis caching strategies

Consistency models

Scalable system design basics

API design best practices

Distributed systems basics

6/5/2026, 10:21:03 AM



System Design Interview Notes

1. Load Balancer

What it is: A load balancer sits between clients and your backend servers. It receives incoming requests and distributes them across multiple server instances so no single server gets overwhelmed.

Why we use it: Without a load balancer, one server handles all traffic — it becomes a bottleneck and a single point of failure. With it, you get high availability, horizontal scalability, and fault tolerance. If one server goes down, the load balancer stops sending traffic to it automatically.

Simple example: You have 3 instances of your Spring Boot Claims service behind an AWS Application Load Balancer. 1000 requests per second arrive — ALB routes ~333 to each instance using round-robin. If instance 2 crashes, ALB detects it via health checks and splits traffic between instances 1 and 3.

Types of Load Balancers:

- **Layer 4 (Transport layer):** Routes based on IP and TCP/UDP port. Very fast, no content inspection. AWS Network Load Balancer (NLB) works at this level. Use when you need ultra-low latency (gaming, financial tick data).
- **Layer 7 (Application layer):** Routes based on HTTP headers, URL paths, cookies, query params. AWS ALB works here. Use when you need path-based routing (`/claims` → service A, `/auth` → service B), host-based routing, or SSL termination.
- **Hardware load balancers:** Physical appliances (F5, Citrix). Expensive, high performance, used in on-prem enterprise setups.
- **Software load balancers:** Nginx, HAProxy, Envoy. Flexible, cheap, widely used in cloud/k8s setups.
- **DNS load balancing:** Multiple IPs returned for the same domain. Simple but no real-time health awareness.
- **Client-side load balancing:** The client holds the list of servers and picks one (used in service meshes, Spring Cloud Ribbon). No central bottleneck.

Load balancing algorithms:

- Round-robin — requests distributed in order
- Least connections — send to server with fewest active connections

- IP hash — same client always hits the same server (useful for session stickiness)
- Weighted round-robin — heavier servers get more traffic

2. Database (SQL vs NoSQL)

What it is: A database is a structured system for persisting and querying data. SQL databases store data in tables with a fixed schema and relationships. NoSQL databases store data in flexible formats — documents, key-value pairs, wide columns, or graphs.

Why we use it: Every application needs persistent storage. The choice between SQL and NoSQL depends on your data shape, query patterns, consistency needs, and scale requirements.

Simple example: In Availity's Claims module — claim metadata (claimId, patientId, status, amount) lives in Aurora MySQL (SQL) because it needs ACID transactions and joins. The claim document payload (full ANSI 837 data, variable fields) lives in MongoDB (NoSQL) because its shape varies by payer and doesn't fit neatly into columns — this is exactly the CQRS pattern you use.

SQL vs NoSQL — Side by Side:

Aspect	SQL	NoSQL
Schema	Fixed, predefined	Flexible, dynamic
Data model	Tables, rows, columns	Document, key-value, wide-column, graph
Relationships	Foreign keys, JOINS	Embedded documents or application-level joins
Transactions	Full ACID	Eventual consistency (usually); some support ACID
Scaling	Vertical (scale up), limited horizontal	Horizontal (scale out) by design
Query language	Standardized SQL	Database-specific APIs
Best for	Financial records, user accounts, orders	Catalogs, event logs, user sessions, social graphs
Examples	MySQL, PostgreSQL, Aurora	MongoDB, Cassandra, DynamoDB, Redis

When to choose SQL: You have relational data, need multi-table transactions, your schema is stable, and correctness > flexibility (billing, claims, payroll).

When to choose NoSQL: Schema evolves frequently, you need massive horizontal scale, data is hierarchical or unstructured, and you can tolerate eventual consistency (product catalogs, activity feeds, sessions).

3. Cache (Redis Basics)

What it is: A cache is a fast in-memory store that holds frequently accessed data so your application doesn't have to hit the database every time. Redis is the most widely used cache — it stores data as key-value pairs in RAM, supports rich data structures (strings, hashes, lists, sets, sorted sets), and can persist data to disk.

Why we use it: Database reads are slow (10–100ms). Redis reads are fast (~0.1ms). For data that doesn't change on every request — user sessions, config, lookup tables, API responses — caching eliminates redundant DB calls, reduces latency, and protects your DB from being overwhelmed.

Simple example: Your Claims service fetches payer configuration on every claim submission. That config rarely changes. Without cache: 1000 claims/sec = 1000 DB reads/sec. With Redis: first request reads from DB and writes to Redis with a 5-minute TTL. Next 999 requests read from Redis — DB load drops to near zero.

```
// Spring Boot example
@Cacheable(value = "payerConfig", key = "#payerId")
public PayerConfig getPayerConfig(String payerId) {
    return payerConfigRepository.findById(payerId).orElseThrow();
}
```

Redis Data Structures:

- **String** — simple values, counters (`SET claim:123:status "SUBMITTED"`)
- **Hash** — object fields (`HSET user:42 name "Thabira" role "SDE"`)
- **List** — ordered sequences, queues
- **Set** — unique values, membership checks
- **Sorted Set** — leaderboard-style ranking with scores

- **TTL** — every key can have an expiry time

Types of Cache:

- **Application cache (in-process):** Cache lives inside the application's memory (e.g., Guava Cache, Caffeine). Ultra-fast, no network hop. Problem: each instance has its own copy — inconsistency across instances.
- **Distributed cache:** Cache lives in a separate shared service (Redis, Memcached). All app instances read from the same cache. Consistent, scalable. Small network latency vs in-process but worth it.
- **CDN cache:** Static assets (images, JS, CSS) cached at edge nodes (CloudFront). Geographically close to users.
- **Database query cache:** Some DBs cache query results internally (MySQL query cache — now deprecated). Limited control.
- **Write-through cache:** Write goes to cache AND DB at the same time. Cache always consistent. Slightly slower writes.
- **Write-back (write-behind) cache:** Write goes to cache first, DB updated asynchronously. Faster writes, risk of data loss if cache crashes.
- **Read-through cache:** Application always reads from cache; cache fetches from DB on miss and populates itself. Transparent to the app.
- **Cache-aside (lazy loading):** App checks cache first. On miss, app reads from DB and populates cache. Most common pattern. You own the logic.

Cache eviction policies (what Redis removes when full):

- **LRU** (Least Recently Used) — evict the key not accessed for the longest time
- **LFU** (Least Frequently Used) — evict the key accessed the fewest times
- **TTL-based** — keys expire on their own schedule
- **No eviction** — Redis returns error when full (good for session stores where you never want silent data loss)

4. Message Queue (Kafka / RabbitMQ)

What it is: A message queue is a system that lets services communicate asynchronously. Producer services write messages to the queue. Consumer services read from it at their own pace. Neither side needs to be online at the same time.

Why we use it: Without a queue, Service A calls Service B directly (synchronous). If B is slow, A is slow. If B is down, A fails. A queue decouples them — A writes to the queue and moves on. B processes when ready. You also get natural buffering (queue absorbs traffic spikes), retry capability, and fan-out (multiple consumers reading the same message).

Simple example: Claim is submitted → Claims service writes a `ClaimSubmitted` event to Kafka → three consumers read it independently: Eligibility service validates coverage, Audit service logs it, Notification service emails the provider. Claims service doesn't wait for any of them.

Kafka vs RabbitMQ:

Aspect	Kafka	RabbitMQ
Model	Log-based, append-only, pull	Message broker, push to consumers
Message retention	Retained for configurable period (days/weeks)	Deleted after acknowledgment
Throughput	Millions of messages/sec	Hundreds of thousands/sec
Consumer model	Consumer group reads at its own offset	Queue-based, message gone once consumed
Replay	Yes — rewind offset to replay old messages	No — consumed = gone
Ordering	Per partition	Per queue
Best for	Event streaming, audit logs, data pipelines, event sourcing	Task queues, RPC, work distribution, routing logic
Use in your stack	Kafka is already in use at Availity	RabbitMQ common in simpler microservice setups

When to use Kafka: You need event streaming, replay, high throughput, audit trails, or multiple independent consumers on the same event.

When to use RabbitMQ: You need task queues with complex routing (topic/fanout/direct exchanges), priority queues, or simpler pub-sub with lower operational overhead.

5. API Gateway

What it is: An API Gateway is a single entry point that sits in front of all your microservices. Every external client request passes through it. It handles routing to the right service, authentication, rate limiting, SSL termination, request/response transformation, logging, and more.

Why we use it: Without a gateway, clients need to know the address of every microservice, every service implements its own auth and rate limiting, and cross-cutting concerns get duplicated everywhere. The gateway centralizes all of that. It also protects your internal services from being exposed directly.

Simple example: A frontend React app calls `/api/claims/submit`. It doesn't know or care that this hits the Claims microservice at `http://claims-svc:8080/internal/claims`. The gateway (Tyk, in your case) receives the request, validates the JWT token, checks rate limits, transforms the path, and proxies to the correct service. The frontend sees one clean API.

What an API Gateway does:

- **Routing** — maps external paths to internal service URLs
- **Authentication / Authorization** — validate JWT, OAuth tokens before the request reaches services
- **Rate limiting** — prevent abuse (100 req/min per API key)
- **SSL termination** — HTTPS ends at the gateway; internal traffic can be HTTP
- **Load balancing** — distribute across service instances
- **Request/response transformation** — add headers, rewrite paths, reshape payloads
- **Logging and tracing** — centralized access logs, correlation IDs injected here
- **Circuit breaking** — stop forwarding to unhealthy services

Examples: AWS API Gateway, Kong, Tyk (your stack), Nginx, Apigee, Envoy.

6. Microservices (Monolithic vs Microservices)

What it is: A monolith is a single deployable unit where all modules (claims, auth, billing, notifications) live in one codebase and are deployed together. Microservices splits the application into small, independently deployable services, each owning its own data and deployed separately.

Why we use it: Monoliths are simple to start with but become painful at scale — one team's change requires redeploying everything, a bug in billing can crash claims, and you can't scale just the busy parts. Microservices let teams own and deploy their services independently, scale only what's needed, and use different tech stacks per service.

Simple example: Availity's Essential Pro is a microservices system. The Claims service, Eligibility service, Auth service, and Notification service are all separate Spring Boot apps deployed as separate containers in Kubernetes. A code change to Claims triggers only the Claims CI/CD pipeline — nothing else restarts.

Monolith vs Microservices:

Aspect	Monolith	Microservices
Deployment	Single unit — all or nothing	Each service deploys independently
Scaling	Scale everything even if only one module is busy	Scale individual services
Development	Simple locally, fast to start	Complex setup, need service discovery, API gateway, etc.
Failure isolation	One bug can bring down the whole app	Failure isolated to one service
Data	Shared database	Each service owns its own database
Team structure	One team owns it all	Each team owns a service
Latency	In-process calls (fast)	Network calls between services (adds latency)
Best for	Early-stage products, small teams	Mature products, large teams, high scale

Microservices challenges to be aware of in interviews:

- Distributed tracing needed (Zipkin, Jaeger, AWS X-Ray)

- Data consistency across services requires Sagas or event-driven patterns
 - Network failures need circuit breakers (Resilience4j)
 - Service discovery needed (Kubernetes DNS, Eureka)
 - Debugging is harder — logs are spread across services
-

Quick-Fire Questions (5–10 lines each)

How does caching work?

Your application first checks the cache (Redis) for the data. If found (cache hit), it returns immediately — no DB call. If not found (cache miss), it reads from the database, stores the result in Redis with a TTL, and returns. On subsequent requests within the TTL window, the cache always hits. The key design decisions are: what to cache (hot data, expensive queries), how long to cache it (TTL), and how to invalidate it when the source changes. Caching works best for read-heavy data that doesn't change often — config tables, reference data, user sessions, API responses.

Why use a message queue?

Message queues decouple the producer from the consumer. The producer writes to the queue and immediately moves on — it doesn't wait for the consumer to process. This means: slow consumers don't block producers, traffic spikes are absorbed by the queue buffer, consumers can retry on failure without the producer knowing, and multiple consumers can process the same message independently. Without a queue, you're doing synchronous REST calls between services — if the downstream service is slow or down, your whole flow breaks.

When there's Kafka for event processing, why implement a queue? What's the need?

Kafka and traditional queues (RabbitMQ, SQS) are complementary, not interchangeable. Kafka is a distributed event log — it's brilliant for high-throughput event streaming, replay, audit logs, and fan-out to many consumers. But Kafka is overkill for simple task distribution. If you have a background job — send an email, resize an image, process a payment — a simple queue like SQS or RabbitMQ is lighter, easier, and has built-in dead-letter queues and visibility timeouts designed for task semantics. Also, Kafka requires

offsets, consumer groups, and partition management — for a job queue with 10 messages/sec, that's unnecessary complexity. Use Kafka when you need streaming and replay. Use a queue when you need reliable task dispatch.

SQL vs NoSQL difference?

SQL databases store data in structured tables with a fixed schema and enforce relationships via foreign keys. They support ACID transactions — critical when correctness matters (banking, claims). NoSQL databases use flexible formats (documents, key-value, wide-column) with no rigid schema — great when data shape varies or evolves. SQL scales vertically (bigger machine) and has limited horizontal sharding. NoSQL scales horizontally by design. Use SQL when your data is relational and transactions span multiple entities. Use NoSQL when you need schema flexibility, massive write throughput, or hierarchical data that doesn't fit rows and columns cleanly.

How to scale a system?

Start vertical (bigger machine) but that has a ceiling. Real scaling is horizontal — add more instances behind a load balancer. Cache aggressively at every layer (Redis for DB reads, CDN for static assets). Add read replicas to split DB read/write load. Partition your database (sharding) when a single DB can't handle volume. Use async message queues to decouple heavy processing from user-facing requests. Use a CDN to serve content from edge nodes near users. Design stateless services so any instance can handle any request — no session affinity needed. And add auto-scaling (Kubernetes HPA, AWS Auto Scaling) so capacity adjusts automatically under load.

Deep-Dive Topics

Redis Caching Strategies

Cache-Aside (Lazy Loading):

The application owns the cache logic. On read: check cache → miss → read DB → write to cache → return. Most common pattern. Cache only contains data actually requested. Downside: first request always misses (cold start), and if cache crashes, all traffic hits DB until it warms up again.

Read-Through:

Cache sits in front of DB. Application only talks to cache. On miss, the cache itself fetches from DB and populates. Transparent to the app. Good for read-heavy systems. Used in providers like AWS ElastiCache with DAX (DynamoDB).

Write-Through:

Every write goes to cache AND DB synchronously. Cache is always consistent with DB. Downside: write latency increases, and you cache data that may never be read again (write amplification). Pairs well with read-through.

Write-Behind (Write-Back):

Write goes to cache immediately; DB is updated asynchronously. Fast writes, low latency. Risk: if Redis crashes before the async flush, you lose data. Use only when you can tolerate occasional data loss (analytics counters, non-critical metrics).

Refresh-Ahead:

Cache proactively refreshes data before TTL expires, based on predicted access patterns. Reduces cache miss latency spikes. Complex to implement correctly — requires knowing what to pre-fetch.

TTL Strategy:

Every cached key should have a TTL. Short TTL = more DB hits but fresher data. Long TTL = fewer DB hits but stale data risk. For critical data (claim status), use short TTL + explicit invalidation on update. For reference data (payer list), longer TTL is fine.

Cache Invalidation:

The hardest problem in caching. Three approaches: TTL expiry (simplest, acceptable staleness), event-driven invalidation (write to DB → publish event → cache listener deletes the key — strong consistency), and versioned keys (append a version to the key, bump on write — old keys naturally expire).

Redis Eviction Policies (when memory is full):

- **allkeys-lru** — evict least recently used key from all keys. Good general default.
- **volatile-lru** — LRU only among keys with TTL set.
- **allkeys-lfu** — evict least frequently used. Better for skewed access patterns.
- **volatile-ttl** — evict key with shortest remaining TTL.
- **noeviction** — reject writes. Safe for session stores where silent loss is unacceptable.

Consistency Models

Consistency in distributed systems describes how up-to-date and synchronized data is across nodes after a write.

Strong Consistency:

After a write completes, every subsequent read from any node sees that write. System behaves as if there's one copy of data. Simple to reason about. Expensive — requires coordination (locks, consensus protocols like Raft/Paxos). Used in: relational databases with synchronous replication, Google Spanner, ZooKeeper. Use when correctness is non-negotiable (financial transactions, inventory).

Eventual Consistency:

After a write, replicas will eventually converge to the same value — but reads during the propagation window may return stale data. No coordination needed — writes are fast. Used in: Cassandra, DynamoDB (default), DNS. Use when availability and performance matter more than instantaneous accuracy (social media likes, shopping cart recommendations, activity feeds).

Read-Your-Writes Consistency:

A specific user always sees their own writes immediately, even if other users might see stale data. Common in user-facing apps — if you update your profile, you see the change instantly. Implemented by routing a user's reads to the replica they last wrote to, or using sticky sessions.

Monotonic Read Consistency:

Once a client reads a value, it never reads an older value for the same key. Prevents the bizarre scenario where you read version 5, then read version 3. Achieved by pinning a client to a specific replica.

Causal Consistency:

Operations that are causally related appear in order to all nodes. If you post a comment in response to a post, anyone who sees the comment also sees the original post. Weaker than strong, stronger than eventual. Used in collaborative tools, distributed databases like MongoDB (causally consistent sessions).

CAP Theorem context:

In a network partition (P), you must choose between Consistency (C) and Availability (A). SQL DBs with synchronous replication choose CP (reject writes if can't confirm all replicas). Cassandra/DynamoDB choose AP (stay available, accept temporary inconsistency). In

practice, network partitions are rare — the real trade-off is latency vs consistency (PACELC theorem).

Scalable System Design Basics

1. Stateless Services:

Keep no session or user state inside the service process. Store state in Redis or a DB. Any instance can serve any request — this is what makes horizontal scaling possible. If your service is stateful (holds in-memory session), you're stuck with sticky sessions and limited scaling.

2. Horizontal Scaling:

Add more instances rather than a bigger machine. Requires a load balancer in front. Works because services are stateless. Combine with auto-scaling (Kubernetes HPA scales pods based on CPU/memory or custom metrics like queue depth).

3. Database Scaling:

- **Read replicas** — offload reads to replicas; primary handles writes only
- **Connection pooling** — services share a pool of DB connections (PgBouncer, HikariCP) to avoid connection exhaustion
- **Sharding** — partition data across multiple DB instances by a shard key (userId, claimId). Each DB owns a subset. Complex but necessary at extreme scale.
- **Caching** — reduce DB load by caching hot reads in Redis

4. Asynchronous Processing:

Move non-user-facing work off the request path. Claim submitted → return 202 Accepted → process eligibility check async via Kafka. Keeps API latency low and decouples heavy processing.

5. CDN for Static Assets:

Never serve images, JS, CSS from your origin servers. Push to CloudFront / Akamai. Edge nodes serve users from the closest geographic location.

6. Database Indexing:

The most impactful single optimization in most systems. Every column used in WHERE, JOIN, ORDER BY should be indexed. Without indexes, queries do full table scans — $O(n)$. With indexes — $O(\log n)$. Over-indexing hurts write performance. Profile your slow query log and add indexes strategically.

7. Pagination:

Never return unbounded result sets. Cursor-based pagination (return a `nextCursor` token) scales better than offset pagination (OFFSET 10000 still scans 10000 rows).

API Design Best Practices

1. Use REST correctly:

- Nouns for resources, not verbs: `/claims` not `/getClaims`
- HTTP methods carry the verb: `GET /claims`, `POST /claims`, `PUT /claims/{id}`, `DELETE /claims/{id}`
- `GET` is idempotent and safe. `PUT` / `DELETE` are idempotent. `POST` is not.
- Use `PATCH` for partial updates, `PUT` for full replacement.

2. Versioning:

Always version your APIs from day one. `/api/v1/claims`. When you need breaking changes, release `/v2` — old clients keep working. Never change a live API's contract without versioning.

3. Status codes — use them correctly:

- `200 OK`, `201 Created`, `202 Accepted` (async processing started)
- `400 Bad Request` (validation failure), `401 Unauthorized` (no/bad token), `403 Forbidden` (token valid, no permission), `404 Not Found`, `409 Conflict`
- `500 Internal Server Error`, `503 Service Unavailable`
- Never return `200` with an error body — that breaks every monitoring tool.

4. Consistent error responses:

Always return a structured error body — not just a status code.

```
{
  "error": "CLAIM_NOT_FOUND",
  "message": "No claim found with id claim-123",
  "timestamp": "2026-06-05T10:00:00Z",
}
```

```
"traceId": "abc-xyz-123"
}
```

5. Idempotency keys:

For POST endpoints that create or mutate (especially payments/claims), accept an **Idempotency-Key** header. If the client retries due to network timeout, the server recognizes the key and returns the original response instead of creating a duplicate.

6. Pagination, filtering, sorting:

Expose these consistently: **GET /claims?**

status=SUBMITTED&page=2&size=20&sort=createdAt,desc. Never return unbounded lists.

7. Rate limiting:

Return **429 Too Many Requests** with a **Retry-After** header. Expose limits in response headers (**X-RateLimit-Limit**, **X-RateLimit-Remaining**).

8. Security basics:

Always validate and sanitize inputs. Use HTTPS only. Authenticate every request (JWT/OAuth2). Authorize by role/scope, not just authentication. Never expose internal error stack traces in API responses.

Distributed Systems Basics

1. Fallacies of Distributed Computing:

The network is NOT reliable. Latency is NOT zero. Bandwidth is NOT infinite. The network is NOT secure. Topology changes. There is NOT one administrator. Transport cost is NOT zero. The network is NOT homogeneous. Every distributed system design must account for these.

2. CAP Theorem:

A distributed system can guarantee at most 2 of 3: Consistency (every read sees the latest write), Availability (every request gets a response), Partition Tolerance (system continues if nodes can't communicate). Since network partitions happen, real systems choose CP or AP. CP: strong consistency, reject requests during partition (ZooKeeper, HBase). AP: stay available, accept eventual consistency (Cassandra, DynamoDB).

3. Consensus:

Getting multiple nodes to agree on a value in the presence of failures. Solved by Raft (used in etcd, CockroachDB) and Paxos. Used for leader election, distributed locks, and replicated

logs. Important to know why you can't just use a DB for distributed locking — you need fencing tokens and lease-based approaches.

4. Idempotency:

An operation that produces the same result whether executed once or many times. Critical in distributed systems where retries happen. Design all mutations to be idempotent — use idempotency keys, natural uniqueness constraints, or event deduplication via Redis **SET NX**.

5. Circuit Breaker:

If Service B is failing, stop sending requests to it — fail fast instead of queueing up. After a timeout, send a test request. If B recovers, close the circuit. Implemented in Spring Boot via Resilience4j. Prevents cascading failure where one slow service takes down the entire call chain.

6. Saga Pattern:

Managing distributed transactions across microservices without a global 2-phase commit. A saga is a sequence of local transactions, each publishing an event. If step 3 fails, compensating transactions undo steps 1 and 2. Two flavors: Choreography (services react to events) and Orchestration (a central saga orchestrator commands each step). Your Claims → Eligibility → Adjudication flow is a candidate for this.

7. Observability (the 3 pillars):

- **Logs** — structured JSON logs with correlation IDs, searchable in ELK/CloudWatch
- **Metrics** — counters, gauges, histograms (Prometheus + Grafana, CloudWatch Metrics)
- **Traces** — distributed traces that follow a request across services (AWS X-Ray, Jaeger, Zipkin)

Without observability, debugging distributed systems is nearly impossible.

8. Health checks and readiness probes:

Every service exposes **/health** (liveness — is the process alive?) and **/ready** (readiness — is it ready to accept traffic?). Kubernetes uses these to restart crashed pods and hold traffic during startup. Spring Boot Actuator gives you these for free.

6/5/2026, 10:23:27 AM



webhooks basics

Grpc Basics

exception handling patterns

performance tuning basics

What happens if traffic increases 10x?

Where will bottleneck occur?

How will data be stored?

What fails first?

How do we scale?

how to Practice thinking like:

"How does this feature scale?"

"What happens if traffic increases 10x?"

"How to debug production issue?"

"How to design fault-tolerant system?"

6/5/2026, 5:19:13 PM



System Design — Round 2 Notes

1. Webhooks Basics

What it is: A webhook is a way for one system to notify another system when something happens, using an HTTP POST request. Instead of the receiver polling ("did anything change?"), the sender pushes a notification the moment the event occurs. It's sometimes called a "reverse API" — the server calls you, not the other way around.

Why we use it: Polling is wasteful. If you poll every 5 seconds to check if a claim status changed, you're making thousands of empty requests. A webhook fires once, exactly when the event happens. Lower latency, lower cost, simpler consumer logic.

Simple example: A payer system processes your claim and its status changes to

ADJUDICATED. It sends an HTTP POST to <https://availability.com/webhooks/claim-status> with the payload `{ "claimId": "C123", "status": "ADJUDICATED" }`. Your service

receives it, updates Aurora, publishes a Kafka event, and notifies the provider. No polling needed.

How to implement webhooks correctly:

- **Receiver must respond fast.** Return `200 OK` immediately. Do not process inline — push to a queue and process async. If you take 10 seconds to respond, the sender will think you failed and retry.
- **Verify the sender.** Senders sign the payload with an HMAC-SHA256 signature using a shared secret. You recompute the signature on your side and compare. Reject if it doesn't match — otherwise anyone can POST fake events to your endpoint.
- **Handle retries and duplicates.** Senders retry on non-200 responses. Your receiver must be idempotent — use the event ID to deduplicate (store processed event IDs in Redis with TTL).
- **Return correct status codes.** `200` = received. `400` = bad payload (don't retry). `500` = server error (do retry). Never return `200` for an invalid payload — you'll silently lose events.
- **Secure the endpoint.** HTTPS only. Validate the `Content-Type`. Rate limit the endpoint. Whitelist sender IPs if the sender publishes them.

Webhook vs Polling vs WebSocket:

	Webhook	Polling	WebSocket
Direction	Server pushes to client	Client pulls from server	Bidirectional, persistent
Latency	Near real-time	Delayed by poll interval	Real-time
Use case	Event notifications between services	Batch status checks	Live dashboards, chat
Complexity	Medium (needs public endpoint, idempotency)	Simple	High (stateful connection)

2. gRPC Basics

What it is: gRPC is a high-performance RPC (Remote Procedure Call) framework built by Google. Instead of REST (text-based JSON over HTTP/1.1), gRPC uses Protocol Buffers (binary serialization) over HTTP/2. You define your service and data types in a `.proto` file, and gRPC auto-generates client and server code in any language.

Why we use it: JSON is human-readable but large and slow to parse. Protobuf is binary — 3–10x smaller and much faster to serialize/deserialize. HTTP/2 gives you multiplexing (multiple requests over one connection), header compression, and streaming. gRPC is ideal for internal service-to-service communication where you control both ends.

Simple example `.proto` file:

```
syntax = "proto3";

service ClaimsService {
  rpc SubmitClaim (ClaimRequest) returns (ClaimResponse);
  rpc StreamClaimUpdates (ClaimRequest) returns (stream ClaimEvent);
}

message ClaimRequest {
  string claim_id = 1;
  string patient_id = 2;
  double amount = 3;
}

message ClaimResponse {
  string status = 1;
  string tracking_id = 2;
}
```

gRPC generates Java stubs from this. Your service implements the interface. The client calls it like a local method — no HTTP path, no JSON parsing, no manual deserialization.

Four communication patterns in gRPC:

- **Unary:** One request, one response. Like REST. `submitClaim(request) → response`
- **Server streaming:** One request, stream of responses. Server pushes multiple messages back. `streamClaimUpdates(request) → stream of ClaimEvent`

- **Client streaming:** Client sends a stream of requests, server returns one response. Useful for bulk uploads.
- **Bidirectional streaming:** Both sides stream simultaneously. Real-time two-way communication. Used in live collaboration, live monitoring.

REST vs gRPC:

Aspect	REST	gRPC
Protocol	HTTP/1.1	HTTP/2
Data format	JSON (text)	Protobuf (binary)
Schema	Optional (OpenAPI)	Mandatory (.proto)
Code generation	Manual or via tools	Auto-generated stubs
Browser support	Native	Needs grpc-web proxy
Streaming	Limited (SSE, WebSocket)	First-class support
Use case	Public APIs, browser clients	Internal microservices

When to use gRPC: Internal microservice communication, polyglot services (Java calling Go calling Python), high-throughput data pipelines, streaming use cases. Not ideal for public APIs where clients are browsers — REST is still the standard there.

Spring Boot gRPC: Use `grpc-spring-boot-starter`. Define `.proto`, annotate your service with `@GrpcService`, inject clients with `@GrpcClient`. Works alongside your existing Spring context.

3. Exception Handling Patterns

Why this matters in interviews: Interviewers want to see that you think beyond the happy path. Good exception handling is the difference between a system that recovers gracefully and one that loses data silently or cascades failures.

Layer 1 — Global Exception Handler (Spring Boot):

Use `@RestControllerAdvice` to centralize all exception handling in one place. No try-catch in every controller.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ClaimNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleNotFound(ClaimNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("CLAIM_NOT_FOUND",
ex.getMessage()));
    }

    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ErrorResponse>
handleValidation(ValidationException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse("VALIDATION_FAILED",
ex.getMessage()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex)
    {
        log.error("Unhandled exception", ex); // log stack trace
here, not in response
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("INTERNAL_ERROR", "Something
went wrong"));
    }
}

```

Never expose stack traces in API responses. Log them internally. Return structured error bodies always.

Layer 2 — Custom Exception Hierarchy:

```

RuntimeException
├─ BusinessException (your base)
│   ├── ClaimNotFoundException      → 404
│   ├── DuplicateClaimException    → 409
│   ├── PayerNotEligibleException  → 422
│   └─ ClaimValidationException    → 400
└─ SystemException
    ├── ExternalServiceException    → 502
    └─ DatabaseException            → 500

```

Typed exceptions let your global handler map cleanly to HTTP status codes without if-else chains. It also makes code readable — `throw new ClaimNotFoundException(id)` is self-documenting.

Layer 3 — Retry Pattern:

Not all failures are permanent. Network blips, DB connection pool exhaustion, and external API timeouts are transient — retry helps. Spring Retry or Resilience4j handles this cleanly.

```

@Retryable(
    value = { ExternalServiceException.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 500, multiplier = 2) // 500ms,
1000ms, 2000ms
)
public EligibilityResponse checkEligibility(String memberId) {
    return eligibilityClient.check(memberId);
}

@Recover
public EligibilityResponse fallback(ExternalServiceException ex,
String memberId) {
    log.error("Eligibility check failed after retries for member
{}", memberId, ex);
    return EligibilityResponse.unknown(); // graceful degradation
}

```

Exponential backoff with jitter prevents thundering herd — if 100 threads all retry at the same moment, you amplify the problem.

Layer 4 — Circuit Breaker (Resilience4j):

```
@CircuitBreaker(name = "eligibilityService", fallbackMethod =
"eligibilityFallback")
public EligibilityResponse checkEligibility(String memberId) {
    return eligibilityClient.check(memberId);
}
```

States: CLOSED (normal) → OPEN (failing, reject immediately) → HALF-OPEN (test one request). Prevents cascading failure. If eligibility service is down, claims service fails fast instead of hanging on every request.

Layer 5 — Kafka Consumer Exception Handling:

```
@KafkaListener(topics = "claim-submitted")
public void handleClaim(ClaimEvent event) {
    try {
        claimProcessor.process(event);
    } catch (RetryableException e) {
        throw e; // Kafka will retry
    } catch (NonRetryableException e) {
        log.error("Non-retryable error, sending to DLT", e);
        deadLetterTemplate.send("claim-submitted.DLT", event);
    }
}
```

Dead Letter Topic (DLT) is your safety net. Failed messages park there for investigation and manual replay. Never let a consumer silently swallow exceptions — you'll lose messages.

Layer 6 — What NOT to do:

- Never catch **Exception** broadly and swallow it silently with just a log line
- Never return **200 OK** with an error body
- Never expose internal error details (DB connection strings, stack traces) to clients
- Never retry non-idempotent operations without idempotency keys

- Never let a circuit breaker fail open without a fallback

4. Performance Tuning Basics

Where to look first: Measure before you optimize. Use profilers, APM tools (Datadog, AWS X-Ray, New Relic), slow query logs, and JVM metrics. Optimizing without measuring is guessing.

Database — the most common bottleneck:

- Add indexes on columns in WHERE, JOIN, ORDER BY. Check `EXPLAIN` / `EXPLAIN ANALYZE` to see if queries use indexes.
- Use connection pooling (HikariCP — already in Spring Boot). Without it, every request opens a new DB connection — DB crashes under load.
- Avoid N+1 queries. Fetching 100 claims then doing 100 separate queries for each claim's payer is N+1. Use JOIN or batch fetch.
- Use read replicas for reporting queries. Never run heavy analytical queries on your primary write DB.
- Paginate all list queries. `SELECT * FROM claims` with no LIMIT on a large table will kill your DB.

JVM / Spring Boot:

- Use async processing (`@Async`, reactive) for I/O-bound operations. Don't block threads waiting for DB or HTTP calls.
- Tune thread pool sizes — too small: requests queue up. Too large: context switching overhead. Profile under realistic load.
- Watch heap size and GC behavior. Frequent full GCs cause stop-the-world pauses and latency spikes. Use G1GC or ZGC for large heaps.
- Avoid creating large objects in hot paths. Object allocation pressure forces more frequent GC.

Caching:

- Cache at the right layer — Redis for DB queries, local Caffeine cache for config/reference data that changes rarely.

- Cache hit rate matters. If hit rate is below 80%, your cache key or TTL strategy is wrong.
- Don't cache mutable, high-cardinality data — you'll waste memory and cause stale data bugs.

HTTP / API layer:

- Enable HTTP keep-alive and connection pooling in your REST clients (RestTemplate/WebClient). Opening a new TCP connection per request is expensive.
- Compress responses with gzip — especially for large JSON payloads. Spring Boot enables this with one config line.
- Use async endpoints for long-running operations. Return **202 Accepted** with a polling URL instead of blocking the HTTP thread.

Kafka:

- Batch producer sends (**linger.ms** , **batch.size**) — small batches mean many network round trips.
- Increase consumer parallelism by adding partitions and consumer instances. One consumer per partition max.
- Monitor consumer lag (offset lag) — if lag grows, consumers can't keep up with producers. Scale out consumers.

Measuring:

- Response time percentiles: P50, P95, P99. Average hides the tail latency that users actually experience.
- Throughput: requests per second at which latency degrades.
- Error rate: any spike in 5xx under load reveals a bottleneck.
- Saturation: CPU %, memory %, DB connections used, thread pool queue depth.

Production Scenario Thinking

What Happens if Traffic Increases 10x?

Walk through each layer of the stack systematically. This is exactly the mental model interviewers want to see.

Layer 1 — Load Balancer:

ALB/Nginx scales automatically in cloud setups. Rarely the bottleneck unless you have a misconfigured single instance. Check: connection limits, SSL termination overhead.

Layer 2 — Application Servers:

Stateless Spring Boot pods behind a load balancer → scale horizontally. Kubernetes HPA auto-adds pods when CPU crosses threshold. What breaks: thread pool exhaustion (incoming requests queue up if all threads are waiting on DB/IO), connection pool exhaustion (if each pod opens 10 DB connections and you go from 5 pods to 50, you need 500 DB connections — does your DB support that?).

Layer 3 — Cache:

Redis is fast and handles high concurrency well. What breaks: cache stampede — if TTL expires and 1000 requests simultaneously miss cache and all hit DB at once. Fix with mutex locking or probabilistic refresh. Also watch: Redis memory limit — if cache is full and eviction policy is wrong, you evict hot keys.

Layer 4 — Database — this fails first, almost always:

DB is stateful and hardest to scale. At 10x traffic: connection pool saturation (add PgBouncer/ProxySQL as a connection pooler between app and DB), slow query volume increases, replication lag builds up on read replicas, lock contention increases on hot rows. Fix: read replicas for read traffic, query optimization, caching hot reads, eventually sharding.

Layer 5 — Message Queue (Kafka):

Producers rarely back up — Kafka absorbs spikes into the log. What breaks: consumer lag — if consumers can't process fast enough, lag grows and processing delay increases. Fix: scale consumers horizontally (add instances up to partition count), increase partition count if you need more parallelism than current partitions allow.

Layer 6 — External Services:

At 10x traffic your call volume to external APIs (payer eligibility endpoints) also increases 10x. External rate limits get hit. Fix: cache external responses where possible, implement circuit breakers, batch calls if the API supports it.

Where the bottleneck occurs — in order of likelihood:

1. Database (connection pool, slow queries, lock contention) — always first

2. External service rate limits
 3. Application thread pool exhaustion
 4. Kafka consumer lag
 5. Cache stampede on key expiry
 6. Load balancer (rare in cloud setups)
-

How Will Data Be Stored at Scale?

Transactional data (claims, users, billing):

Start with a single Aurora MySQL primary + read replicas. As write volume grows: vertical scaling first (larger RDS instance), then read replicas for read traffic, then consider Aurora Serverless for variable workloads. At extreme scale: sharding by `claimId` hash or `tenantId`. Each shard is an independent DB — routing logic at application layer.

Event/audit data:

Never store high-volume event logs in your transactional DB. Write to Kafka → consume → store in S3 (cheap, durable) or Elasticsearch (searchable). Use columnar formats (Parquet) for analytical queries on S3 via Athena.

Document data (variable schema):

MongoDB scales horizontally via sharding. Shard key choice matters — poor shard key (low cardinality, like `status`) creates hot shards. Good shard key distributes writes evenly (`claimId` or compound key).

Session / cache data:

Redis Cluster for horizontal scaling. Data partitioned automatically across nodes. Add Redis Sentinels or use Redis Cluster mode for high availability.

Cold data:

Data older than 90 days rarely accessed — archive to S3 Glacier or move to a separate analytics DB. Don't let your transactional DB grow unbounded.

What Fails First?

In almost every system under load stress:

1. **Database connections** — pool exhausts, new requests hang waiting for a connection

2. **Slow queries** — under 10x read load, unindexed queries that were "fast enough" at 1x become table scans on a hot DB
3. **External API rate limits** — third-party payer APIs don't scale with you
4. **Application thread pool** — if threads are blocked on slow DB/IO, incoming HTTP requests queue up and eventually timeout
5. **Memory pressure / GC pauses** — 10x load means 10x object allocation rate; GC pauses increase, latency spikes appear in P99
6. **Kafka consumer lag** — consumers fall behind producers if downstream processing (DB writes, external calls) is slow

The pattern is always: **stateful systems fail before stateless ones**. Your Spring Boot service is easy to scale. Your DB is not.

How Do We Scale?

Immediate (horizontal scaling):

Add more application instances. Kubernetes HPA does this automatically. Works instantly for stateless services.

Short-term (caching and query optimization):

Add Redis caching for hot DB reads. Add indexes on slow queries. Add DB read replicas and route read traffic there. These steps typically handle 5–10x without architectural changes.

Medium-term (async and decoupling):

Move synchronous flows to async (Kafka). Return 202 immediately, process in background. This absorbs traffic spikes into queue buffers and gives you time to process at your own rate.

Long-term (data tier scaling):

Connection pooler (PgBouncer) in front of DB. DB sharding for write-heavy workloads. CQRS — separate write model (normalized, transactional) from read model (denormalized, fast). Archive old data to S3. Use Elasticsearch for search instead of **LIKE** queries on MySQL.

Architectural (if you outgrow everything above):

Multi-region active-passive or active-active. Global load balancing. Data partitioned by geography. Cell-based architecture (each cell handles a subset of tenants and fails independently).

How to Think Like a Senior Engineer

Mental Model 1: "How Does This Feature Scale?"

Every time you build or review a feature, run it through this checklist before you consider it done:

Ask about the data:

- How many records exist today? In 1 year at 10x growth?
- Is there a query that touches all of them (no WHERE clause, no index)?
- Is there a table that gets written to on every user action?

Ask about the read/write ratio:

- Is this read-heavy (serve from cache/read replica) or write-heavy (need partitioning)?
- Can reads tolerate slight staleness, or does the user need the freshest data?

Ask about concurrency:

- What happens if 100 users do this simultaneously?
- Is there a shared lock, a shared counter, a shared row they all update?
- Race condition if two requests process the same claimId at the same time?

Ask about failure:

- If the downstream service is down, what happens to this request?
- Is there a retry? Is it idempotent?
- Does a failed request leave data in a partial state?

Practical habit: Before every PR, write one line in the PR description: "Scaling concern: X". Even if the answer is "none identified", the act of asking trains the instinct.

Mental Model 2: "What Happens if Traffic Increases 10x?"

Walk the call stack from browser to DB and ask at each layer: "Can this handle 10x?"

Browser/Client

↓ × 10

CDN / API Gateway → Can it route 10x? Usually yes (managed service)

↓ × 10

Load Balancer → Can it distribute 10x? Usually yes

↓ × 10

Application Server (pods) → Can it handle 10x? Add pods (HPA)

↓ × 10

Cache (Redis) → Can it serve 10x reads? Usually yes, watch stampede

↓ (only cache misses, maybe 2x of original)

Database → CAN IT? This is where you slow down and think.

↓ × 10

External APIs → Rate limits? Timeout budgets?

The DB question breaks into:

- Connections: 10x pods × connections per pod = does max_connections allow this?
- Query time: does EXPLAIN show a full table scan that's acceptable at 1x but kills you at 10x?
- Lock contention: is there a hot row (a status counter, a sequence, a shared config row) that serializes writes?
- Replication lag: at 10x writes, do read replicas fall behind and serve stale data?

Answering these for any system — even hypothetically — is exactly what system design interviewers want to hear.

Mental Model 3: "How to Debug a Production Issue?"

Follow this structured approach every time. Don't guess. Narrow.

Step 1 — Define the symptom precisely.

"Something is slow" is not actionable. "P99 latency on `POST /claims` spiked from 200ms to 4s at 14:32 IST, error rate is 0%, CPU is normal" is.

Step 2 — Establish the timeline.

When did it start? Was there a deployment, a config change, a traffic spike, or a cron job that ran? Correlation with changes is the fastest path to root cause.

Step 3 — Check the 4 golden signals:

- Latency (is it slow?)
- Traffic (is volume normal, higher, lower?)
- Errors (what's the error rate and which errors?)
- Saturation (CPU, memory, DB connections, thread pool — what's at capacity?)

Step 4 — Isolate the layer.

Is it slow at the API Gateway? Then it's not the app. Slow at the app but DB queries are fast? It's the app logic or thread pool. DB queries slow? Check EXPLAIN and slow query log. External service calls slow? Check circuit breaker metrics.

Step 5 — Follow the trace.

Distributed tracing (X-Ray, Jaeger) shows you which span in the request took the time. A 4-second request might be: 10ms Spring logic + 3900ms DB query + 90ms response serialization. The trace tells you exactly where.

Step 6 — Form a hypothesis, verify, fix.

"I think this query on `claims` table has no index on `submitted_date` and is doing a full scan under high load." → Check `EXPLAIN`. → Confirm. → Add index. → Deploy. → Confirm latency drops.

Step 7 — Write a postmortem.

Timeline, root cause, impact, fix, and — most importantly — what systemic change prevents this class of issue from recurring. This is what separates senior engineers from mid-level engineers.

Mental Model 4: "How to Design a Fault-Tolerant System?"

Fault tolerance means the system continues to function (possibly in a degraded mode) when components fail. The core principle: **assume everything fails, design for it explicitly.**

Eliminate single points of failure:

Every component that exists as a single instance is a SPOF. Multiple app instances behind a

load balancer. Multi-AZ DB (Aurora automatically replicates across AZs). Redis with Sentinel or Cluster mode. Kafka with replication factor ≥ 3 . If any component going down takes your system with it, that's a SPOF to fix.

Graceful degradation:

When a non-critical dependency fails, return a degraded response rather than an error. Eligibility service is down → accept the claim, mark it pending eligibility check, process later. Notification service is down → claim still submits successfully, notification retried via Kafka DLT. Users should never see a hard failure for a background service's downtime.

Timeouts everywhere:

Every external call — DB query, HTTP call, Redis call — needs an explicit timeout. Without timeouts, a slow dependency holds your thread forever, thread pool exhausts, and your service appears down even though it's alive. Set connect timeout, read timeout, and circuit breaker threshold for every integration.

Idempotent operations:

Design mutations to be safely retried. Use idempotency keys on claim submissions. Make Kafka consumers idempotent — receiving the same event twice should not create duplicate data. This lets you retry aggressively without fear of data corruption.

Bulkhead pattern:

Separate thread pools for separate concerns. Your thread pool for external payer API calls should not share threads with your claim submission flow. If payer API is slow and saturates its thread pool, claim submissions are unaffected. This is the bulkhead — like watertight compartments in a ship.

Test your failure modes:

Chaos engineering — deliberately kill instances, inject latency, drop network packets — and verify the system behaves correctly. If you only find out your circuit breaker is misconfigured during a real outage, that's too late. Netflix's Chaos Monkey approach, even informally, reveals assumptions you didn't know you were making.

The three questions for every design decision:

1. What happens when this component is down?
2. What happens when this component is slow?
3. What happens when this component returns corrupt/unexpected data?

If you can answer all three with "the system degrades gracefully and recovers", the design is fault-tolerant.

6/5/2026, 5:21:26 PM



Database Partitioning — Complete Guide

What is Partitioning?

Partitioning means splitting a large table into smaller, manageable pieces. The data is divided across multiple physical storage units, but the application queries it as if it's one table. The DB engine (or your application) figures out which partition to read from or write to.

Why partition:

- A table with 500 million rows is slow to query even with indexes — the index itself becomes huge
- Writes on a single table create lock contention
- Archiving old data becomes hard when everything is in one table
- Backup and maintenance operations lock the whole table

Two Broad Categories

Partitioning

- ├ Vertical Partitioning → split **by** columns
- └ Horizontal Partitioning → split **by** rows (**this is** what people mean **99%** of the time)
 - ├ Range Partitioning
 - ├ List Partitioning
 - ├ Hash Partitioning
 - ├ Key Partitioning
 - └ Composite Partitioning

Vertical Partitioning

Split a wide table into narrower tables by column group. All rows stay together, but columns are separated.

Example: Your `claims` table has 40 columns — 8 are queried on every read (hot columns), 32 are rarely accessed audit/detail columns.

```
claims table (40 cols, queried millions/day)
    ↓ vertical partition
claims_core (claimId, patientId, status, amount, payerId,
createdAt, updatedAt, tenantId)
claims_detail (claimId, diagnosisCodes, procedureCodes,
renderingProvider, billingNotes, ...)
```

Why: `claims_core` fits more rows per DB page → fewer I/O operations → faster reads.

`claims_detail` is fetched only when needed (claim detail screen, not list screen).

This is essentially normalization. You already do a form of this in your CQRS setup — write model has full detail, read model has projected summary.

Horizontal Partitioning (Sharding the table)

Split by rows. Each partition holds a subset of rows based on a partition key. Same columns everywhere, different data.

1. Range Partitioning

Divide rows based on a range of values in a column — most commonly a date column.

```
-- MySQL example
CREATE TABLE claims (
    claimId      VARCHAR(36),
    submittedAt  DATE,
    status       VARCHAR(20),
    amount       DECIMAL(10, 2)
)
PARTITION BY RANGE (YEAR(submittedAt)) (
    PARTITION p2022 VALUES LESS THAN (2023),
```

```
PARTITION p2023 VALUES LESS THAN (2024),
PARTITION p2024 VALUES LESS THAN (2025),
PARTITION p2025 VALUES LESS THAN (2026),
PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

Query behavior: `SELECT * FROM claims WHERE submittedAt BETWEEN '2024-01-01' AND '2024-12-31'` — DB only scans partition `p2024`. This is called **partition pruning**.

Best for: Time-series data, audit logs, billing cycles, anything with date-based queries and archival needs.

Pros:

- Natural fit for date-range queries
- Easy to archive — `ALTER TABLE claims DROP PARTITION p2022` deletes a whole year instantly, no DELETE statement scanning millions of rows
- Partition pruning makes range queries very fast

Cons:

- Hot partition problem — all current writes go to the latest partition. 2025 partition gets hammered, older ones are idle.
- If your queries are not date-range based, no benefit

2. List Partitioning

Divide rows based on a specific set of discrete values — like region, status, or tenant.

```
CREATE TABLE claims (
  claimId VARCHAR(36),
  region  VARCHAR(20),
  status  VARCHAR(20),
  amount  DECIMAL(10,2)
)
PARTITION BY LIST COLUMNS (region) (
  PARTITION p_east VALUES IN ('US-EAST-1', 'US-EAST-2'),
```

```

PARTITION p_west    VALUES IN ('US-WEST-1', 'US-WEST-2'),
PARTITION p_central VALUES IN ('US-CENTRAL-1'),
PARTITION p_india   VALUES IN ('AP-SOUTH-1')
);

```

Best for: Multi-tenant systems partitioned by tenantId, regional data isolation, partitioning by a known fixed set of categories.

Pros:

- Clean data isolation per category
- Regulatory compliance — Indian tenant data physically stays in p_india partition

Cons:

- Adding new values requires ALTER TABLE to add a partition
- Uneven data distribution if some regions have far more data

3. Hash Partitioning

Apply a hash function to the partition key. Row goes to partition `hash(key) % N`.

```

CREATE TABLE claims (
  claimId  BIGINT,
  status   VARCHAR(20),
  amount   DECIMAL(10,2)
)
PARTITION BY HASH (claimId)
PARTITIONS 8;
-- DB automatically routes claimId=1001 to partition (1001 % 8) =
partition 1

```

Best for: When you have no natural range or list grouping but need to spread load evenly. High-write tables where you want to distribute write contention across partitions.

Pros:

- Very even data distribution — no hot partitions

- No need to define partition ranges upfront

Cons:

- Partition pruning only works if you query by exact claimId (the hash key). Range queries (`WHERE claimId BETWEEN 100 AND 200`) scan all partitions — no benefit.
- Hard to archive — data from all time periods is mixed across all partitions
- If you change the number of partitions (8 → 16), you have to rehash everything — massive data migration

4. Key Partitioning

Similar to hash partitioning but the DB engine provides the hash function (usually MySQL's internal hash). You just specify the column.

```
PARTITION BY KEY (claimId)
PARTITIONS 8;
```

Works well with primary keys. Less control than hash but simpler to set up.

5. Composite (Sub) Partitioning

Combine two strategies. Partition by range first, then hash within each range partition.

```
PARTITION BY RANGE (YEAR(submittedAt))
SUBPARTITION BY HASH (claimId)
SUBPARTITIONS 4 (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026)
);
```

Each year partition is further split into 4 hash sub-partitions. You get both date-range pruning AND even write distribution within the year.

Best for: Very large tables where a single strategy isn't enough. Common in data warehouses and analytics systems.

Sharding — Application-Level Horizontal Partitioning

When your table grows beyond what one DB server can hold, sharding distributes partitions across **multiple physical database servers**. This is the big leap — from partitioning within one DB to partitioning across many DBs.

Single DB (one server)

```
|— claims_p2023
|— claims_p2024
└— claims_p2025
```

Sharded across servers:

```
DB Server 1 → Shard A (tenantId 0000-3333)
DB Server 2 → Shard B (tenantId 3334-6666)
DB Server 3 → Shard C (tenantId 6667-9999)
```

Shard key selection — the most critical decision:

A bad shard key ruins everything. Criteria for a good shard key:

- **High cardinality** — enough distinct values to distribute data evenly
- **Even write distribution** — no hot shard (one shard getting all writes)
- **Query locality** — most queries should hit one shard, not all of them
- **Immutable** — once a row is sharded, changing the shard key means migrating the row

Common shard keys:

Shard Key	Good for	Risk
<code>tenantId</code>	Multi-tenant SaaS	Large tenants create hot shards
<code>userId</code>	Social platforms	Celebrity users (high-follower accounts) skew shards
<code>claimId</code> hash	Even distribution	Cross-shard queries become scatter-gather
<code>geographyId</code>	Regional compliance	Uneven regional traffic
<code>createdAt</code>	Time-series	Recent shard always hot

Consistent hashing: Used to avoid rehashing everything when you add a shard. Instead of `hash(key) % N`, keys are mapped to a ring and each server owns a range of the ring. Adding a server only moves a fraction of keys, not all of them. Used in Cassandra, DynamoDB, Redis Cluster.

Partitioning Strategies Comparison

Strategy	Split by	Best query pattern	Hot partition risk	Archive friendly
Range	Value ranges (date)	Date range queries	Yes — latest range	Yes — drop old partition
List	Discrete values (region)	Exact category filter	Maybe — uneven categories	Partial
Hash	Hash of key	Point lookups by key	No — even distribution	No — data mixed
Composite	Range + Hash	Date range + point lookup	Low	Yes
Sharding	Across DB servers	Depends on shard key	Depends on shard key	Depends

Partitioning in Your Stack (Practical Relevance)

Aurora MySQL: Supports all MySQL partition types natively. For your Claims table, range partitioning by `submittedAt` is the most natural fit — queries are almost always date-bound, and you can drop old year partitions for archival instead of running expensive DELETE jobs.

MongoDB: Sharding is built-in and first-class. You define a shard key when creating a collection. Mongo's balancer automatically moves chunks between shards. For claims documents, `{ tenantId: 1, claimId: 1 }` as a compound shard key gives you tenant isolation and even distribution.

Kafka: Partitioning in Kafka is also hash-based on the message key. `claimId` as the Kafka message key ensures all events for the same claim go to the same partition — preserving order per claim. This is important when consumers process claim lifecycle events (SUBMITTED → VALIDATED → ADJUDICATED) and order matters.

Common Interview Q&A

Q: What's the difference between partitioning and sharding?

Partitioning splits data within a single DB server. Sharding splits data across multiple DB servers. Sharding is application-level (your code decides which server to query); partitioning is DB-engine-level (the engine handles it transparently).

Q: What is a hot partition and how do you fix it?

A hot partition is one that receives disproportionately more reads or writes than others — defeating the purpose of partitioning. Happens with range partitioning on date (current date always hot) or list partitioning on uneven categories. Fix: switch to hash partitioning for write distribution, add caching in front of hot reads, or use consistent hashing with virtual nodes.

Q: When would you NOT partition?

When your table is small (under a few million rows with good indexes, partitioning adds overhead with no benefit). When your queries don't align with the partition key (you'd scan all partitions anyway). When operational complexity isn't worth it for your scale.

Q: Cross-shard queries — how do you handle them?

Avoid them by design — choose a shard key that keeps related data together. When unavoidable (reporting, admin queries), use scatter-gather: query all shards in parallel, merge results in the application layer. Or maintain a separate read model (Elasticsearch, data warehouse) that aggregates across shards for reporting use cases.

6/6/2026, 6:51:40 AM